

EV Regenerative Braking Energy Recovery: A Computational Investigation

Project 3 — Future Electric Vehicle Systems

Project 3 — Future Electric Vehicle Systems

Abstract

Regenerative braking is one of the main ways an electric vehicle can recover part of the energy that would otherwise be lost during braking. In this project, I developed a Python-based physics simulator to study this process using basic mechanical-energy equations.

The model calculates the kinetic energy removed during braking, estimates the recovered energy using a selected regenerative efficiency, calculates braking distance, and tracks the change in battery state of charge (SOC). I then used the model for a set of controlled experiments covering vehicle speed, vehicle mass, regenerative efficiency, battery capacity and deceleration.

For the baseline case, an 1800 kg vehicle with a 60 kWh battery, 80% regenerative efficiency and a starting SOC of 40% was tested through five braking events. The simulation recovered approximately 0.3241 kWh, giving a final modeled SOC of 40.5401%.

The results follow the expected relationships from the equations: recovered energy increases with the square of speed, increases linearly with mass and regenerative efficiency, while the SOC increase from a fixed amount of recovered energy becomes smaller as battery capacity increases. The project is a simplified computational model and is not intended to reproduce every condition of a real EV.

Keywords

Electric vehicle; regenerative braking; kinetic energy; energy recovery; battery SOC; Python simulation; computational engineering.

1. Introduction

When an electric vehicle slows down, its kinetic energy has to be removed. In a conventional braking system, much of this energy is released as heat. Regenerative braking provides another possibility: part of the vehicle's kinetic energy can be converted back into electrical energy and returned to the battery.

The amount of energy that can be recovered depends on the vehicle's condition and on the efficiency of the regenerative system. Speed is especially important because kinetic energy depends on the square of velocity. Vehicle mass and regenerative efficiency also have direct effects on the calculated recovery.

I developed this project to study these relationships through a simple Python model rather than trying to build a physical braking system. The aim was to keep the model understandable while still allowing several different

conditions to be tested.

2. Problem Statement

The project investigates how much energy can be recovered when an electric vehicle slows down and how changes in basic vehicle parameters affect that result.

The model focuses on the relationships between speed, mass, braking conditions, regenerative efficiency and battery capacity.

3. Objectives

The objectives of the project were:

1. Build a mathematical model for regenerative braking.
2. Implement the model in Python.
3. Calculate the kinetic energy available during braking.
4. Estimate the energy recovered through regenerative braking.
5. Calculate braking distance.
6. Track battery SOC over repeated braking events.
7. Study the effect of speed, vehicle mass and regenerative efficiency.
8. Study how battery capacity affects the resulting SOC increase.

4. Mathematical Model

The kinetic energy of the moving vehicle is:

$$E = 1/2 m v^2$$

When the vehicle slows from initial velocity v_i to final velocity v_f , the kinetic energy removed is:

$$E_{\text{braking}} = 1/2 m (v_i^2 - v_f^2)$$

The simplified model assumes that a fixed fraction η of this energy is recovered:

$$E_{\text{recovered}} = \eta E_{\text{braking}}$$

The program converts the recovered energy from joules to kWh.

For constant deceleration, braking distance is calculated using:

$$d = (v_i^2 - v_f^2) / (2a)$$

The change in battery SOC is calculated from:

$$\Delta \text{SOC} = \text{recovered energy} / \text{battery capacity} \times 100$$

All speeds entered in km/h are converted to m/s before these calculations are performed.

5. Computational Method

The simulator takes the vehicle parameters and braking conditions as inputs. For each braking event it converts the speed, calculates the initial and final kinetic energy, finds the braking energy, applies regenerative efficiency and converts the result to kWh.

For the multi-event test, the recovered energy from each event is added to the previous SOC value.

The baseline parameters are:

The five braking events are:

6. Python Implementation

The simulator is written in Python. The calculations are kept separate from the graphs so that the same model can be used for each experiment.

The main program is available in simulator.py.

The program generates the numerical datasets in the data folder and the figures in the figures folder.

7. Computational Experiments

Experiment 1 — Initial speed

The initial speed was changed from 40 to 100 km/h while keeping the vehicle mass at 1800 kg, the final speed at 0 km/h and the regenerative efficiency at 80%.

Figure 1. Initial speed vs recovered energy.

The recovered energy rises increasingly quickly as speed increases. This agrees with the kinetic-energy equation because energy is proportional to the square of speed.

Experiment 2 — Vehicle mass

The mass was changed from 1000 to 2700 kg while braking from 80 to 40 km/h at 80% efficiency.

Figure 2. Vehicle mass vs recovered energy.

The result is approximately linear. With the speed change kept constant, a heavier vehicle has proportionally more kinetic energy available for recovery.

Experiment 3 — Regenerative efficiency

Efficiency was changed from 40% to 100% for an 1800 kg vehicle slowing from 80 to 40 km/h.

Figure 3. Regenerative efficiency vs recovered energy.

The result is linear because the model directly multiplies braking energy by the efficiency value.

Experiment 4 — Battery capacity

The total recovered energy from the five-event baseline simulation was kept fixed while battery capacity was changed.

Figure 4. Battery capacity vs SOC increase.

A smaller battery shows a larger percentage SOC increase for the same recovered energy. This experiment changes the SOC result rather than the kinetic energy available during braking.

Experiment 5 — Deceleration and braking distance

Initial speed was fixed at 80 km/h and final speed at 0 km/h. Deceleration was varied.

Figure 5. Deceleration vs braking distance.

The calculated braking distance decreases as deceleration increases. This follows the braking-distance equation used in the model.

8. Five-Event Simulation

The baseline five-event simulation gives:

Figure 6. Recovered energy per braking event.

The total recovered energy is approximately 0.3241 kWh.

The battery SOC increases from 40% to approximately 40.5401%.

Figure 7. Battery SOC during the five braking events.

9. Results and Observations

The experiments show several clear relationships.

First, speed has the strongest nonlinear effect in this simplified model because kinetic energy contains v^2 . This means that a higher-speed braking event can contain much more recoverable energy than a similar event at a lower speed.

Second, vehicle mass has a direct linear effect when the speed change is kept constant.

Third, regenerative efficiency has a direct linear effect because the model uses efficiency as a multiplier on the available braking energy.

Finally, battery capacity affects the size of the SOC increase. A smaller battery shows a larger percentage increase when the same amount of recovered energy is added.

10. Sensitivity Analysis

The main relationships can be summarized as:

These relationships come directly from the equations used in the simulator.

11. Model Check

The Python results can be checked against the equations independently. For each braking event, the initial and final kinetic energies are calculated first, followed by the braking energy and recovered energy.

The same equations are then used by the Python program. Agreement between the hand-calculated equation and the program output provides a basic check that the implementation follows the intended mathematical model.

12. Discussion

The project shows that a small set of basic mechanics equations can be turned into a useful computational engineering model.

The five-event simulation demonstrates cumulative recovery, while the parameter experiments make it possible to see how individual inputs affect the result.

The most important result is the strong effect of speed. Since kinetic energy depends on the square of velocity, increasing speed produces a disproportionately large increase in the energy available during braking.

The results should still be treated as model outputs rather than measurements from a real EV. Real regenerative braking involves additional losses and operating limits that are not represented here.

13. Limitations

The model is intentionally simplified.

It does not separately model motor and inverter losses, battery temperature, maximum charging power, SOC-dependent charging limits, tire traction, motor torque characteristics or battery degradation.

The regenerative efficiency is treated as a fixed value, and the braking-distance calculation assumes constant deceleration.

No physical EV measurements were collected, so the project does not provide experimental validation.

14. Conclusion

A Python-based regenerative-braking simulator was developed and used to study energy recovery in a simplified electric-vehicle model.

The baseline five-event simulation recovered approximately 0.3241 kWh and increased the modeled battery SOC from 40% to 40.5401%.

The additional experiments showed the expected relationships between recovered energy and vehicle speed, mass and regenerative efficiency. The battery-capacity experiment also showed how the same recovered energy produces different SOC increases in batteries of different sizes.

Overall, the project demonstrates how basic mechanics and energy equations can be converted into a computational engineering investigation. The model is simple, but it provides a clear starting point for studying regenerative braking and for later extensions.

15. Future Scope

The model could later be extended using real driving-cycle data, motor/inverter efficiency maps, battery charging constraints, temperature effects, CSV-based workflows and an interactive interface. These would make the simulation closer to the behaviour of a real EV.

References

1. Python Software Foundation. Python documentation.
2. Matplotlib Development Team. Matplotlib documentation.
3. Project 3 source code and computational results.

Appendix — Source Code

The complete Python implementation is provided separately in `simulator.py`.

The numerical outputs used for the paper are provided as CSV files in the data folder.